Dr. M.G.R.

Educational and Research Institute

(Deemed to be University)

(Declared under Section 3 of UGC Act 1956)

AI-POWERED SMART HELMET WITH ACCIDENT PREDICTION AND EMERGENCY ALERT SYSTEM

A MINI PROJECT REPORT

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Technology
in
Computer Science and Engineering (Artificial Intelligence and Data Science)

Department of Computer Science and Engineering (AI & DS)

Submitted by:

KARTHICK S

MIDHUN SATHISHKUMAR

KAMESH KUMAR J

Academic Year: 2025 - 2026

Dr. M.G.R.

EDUCATIONAL AND RESEARCH INSTITUTE

(Deemed to be University)

Department of Computer Science and Engineering (AI & DS)

BONAFIDE CERTIFICATE

This is to certify that the Mini Project entitled "AI-POWERED SMART HELMET WITH ACCIDENT PREDICTION AND EMERGENCY ALERT SYSTEM" is a bonafide work carried out by Karthick S, Midhun Sathishkumar, and Kamesh Kumar J in partial fulfillment of the requirements for the award of the degree of Bachelor of Technology in Computer Science and Engineering (Artificial Intelligence and Data Science) from Dr. M.G.R. Educational and Research Institute during the academic year 2025 - 2026.

Internal Guide	**Head of Department**
Name: ____________________	Name: ____________________
Signature: ________________	Signature: ________________

ACKNOWLEDGEMENT

We would like to express our sincere gratitude to our project guide and the faculty of the Department of Computer Science and Engineering (AI & DS) at Dr. M.G.R. Educational and Research Institute for their continuous support, encouragement, and valuable guidance throughout the course of this mini project.

We extend our heartfelt thanks to the Head of the Department for providing us with the necessary resources and infrastructure to carry out this project successfully.

We are also grateful to our friends and family for their moral support and motivation during the development of this project.

Finally, we thank all the open-source communities and documentation authors whose tools, libraries, and resources made this project possible.

Karthick S

Midhun Sathishkumar

Kamesh Kumar J

ABSTRACT

Road accidents involving two-wheeler vehicles constitute a significant proportion of global traffic fatalities. Key contributing factors include overspeeding, riding under the influence of alcohol, rider drowsiness, and delayed emergency medical response during the critical "Golden Hour." Traditional helmets provide only passive physical protection and lack intelligent features for proactive safety monitoring. This project presents an AI-Powered Smart Helmet with Accident Prediction and Emergency Alert System, a full-stack software prototype that demonstrates the integration of Artificial Intelligence, Internet of Things (IoT), and Machine Learning technologies for motorcycle rider safety. The system employs a Random Forest Classifier trained on synthetic sensor data to predict rider conditions across three classes: Safe, At-Risk, and Danger.

The current prototype is a 50% Working Software Prototype that uses simulated sensor data to demonstrate the complete system pipeline. The backend is built using Python FastAPI, which processes sensor telemetry, executes real-time ML predictions, and manages emergency alert dispatching. The frontend is a modern React.js web dashboard built with Vite, providing live telemetry visualization, AI prediction panels, historical data charts, and an emergency SOS alert card with GPS location and Google Maps integration. The system supports five simulation modes (Safe, Risky, Drunk, Drowsy, and Crash), each demonstrating distinct rider conditions with corresponding sensor value patterns and AI predictions. In crash mode, the system automatically triggers an emergency SOS alert with GPS coordinates and simulated GSM SMS dispatch to emergency contacts.

Real hardware integration using ESP32 microcontrollers, MPU6050 accelerometers, MQ-3 alcohol sensors, NEO-6M GPS modules, and SIM800L GSM modules is planned as future scope to evolve this software prototype into a complete physical product.

Keywords: *Smart Helmet, Accident Prediction, Emergency Alert, Random Forest, IoT, Machine Learning, Road Safety, GPS Tracking, GSM Alert*

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF TABLES

CHAPTER 1: INTRODUCTION

1.1 Overview

The rapid expansion of two-wheeler usage in urban and rural areas has led to a corresponding increase in road accidents, particularly involving motorcycles. According to the World Health Organization (WHO), road traffic injuries are the leading cause of death among young people aged 15 to 29 years, and two-wheeler riders constitute a disproportionately large share of these casualties. Traditional helmets, while effective in providing physical protection to the skull, offer no intelligent capabilities for accident prediction, rider condition monitoring, or emergency communication.

This project, titled "AI-Powered Smart Helmet with Accident Prediction and Emergency Alert System," presents a technology-driven solution that transforms an ordinary helmet into an intelligent safety device. By integrating multiple sensors, a Machine Learning prediction engine, and an automated emergency alert mechanism, the system aims to significantly reduce accident-related fatalities by enabling proactive safety monitoring and rapid emergency response.

The current implementation is a 50% working software prototype that demonstrates the complete data pipeline - from sensor data acquisition (simulated) to AI-based risk prediction and emergency alert dispatch - via a live web-based dashboard. Real hardware integration is planned as future scope.

1.2 Road Safety and Two-Wheeler Accident Problem

Two-wheeler vehicles account for a significant percentage of overall traffic on Indian roads. The National Crime Records Bureau (NCRB) data indicates that India witnesses a disproportionately high number of road accident fatalities involving motorcycle riders compared to other vehicle categories. The primary causes include:

- Overspeeding: Exceeding safe speed limits, especially on highways and poorly maintained roads, leads to loss of vehicle control.
- Alcohol Impairment: Riding under the influence of alcohol severely compromises judgment, reaction time, and motor coordination.

• Rider Fatigue and Drowsiness: Long-distance riders often experience microsleep episodes that lead to sudden loss of vehicular control.

• Delayed Emergency Response: In many accident scenarios, especially in remote areas or during nighttime, there is a significant delay between the accident occurrence and the arrival of medical help. This delay, often exceeding the critical "Golden Hour," substantially reduces survival rates.

1.3 Need for Smart Helmet System

The limitations of traditional helmets highlight the urgent need for a smart helmet system that can:

• Continuously monitor rider parameters such as speed, body orientation, blood alcohol levels, and fatigue levels.

• Predict potential dangers before they escalate into accidents by analyzing sensor patterns using Artificial Intelligence.

• Automatically alert emergency services and family members with precise GPS location data in the event of a crash, without requiring manual intervention.

• Prevent intoxicated riding by locking the vehicle ignition system when unsafe alcohol levels are detected.

1.4 Role of AI and IoT in Rider Safety

Artificial Intelligence (AI) enables the system to analyze multi-dimensional sensor data and classify rider conditions in real-time. Rather than relying on simple threshold-based rules, AI models such as Random Forest classifiers can learn complex non-linear relationships between sensor readings and safety outcomes, providing more accurate and reliable predictions.

Internet of Things (IoT) provides the framework for connecting physical sensors (accelerometers, alcohol sensors, GPS modules) to cloud-based processing systems via wireless communication protocols. IoT architecture enables real-time data streaming, remote monitoring, and automated response mechanisms.

The combination of AI and IoT in this project creates an intelligent safety ecosystem where sensors continuously collect rider and environmental data, AI algorithms process this data to predict risk levels, IoT communication channels transmit alerts to emergency services, and a web dashboard provides real-time visibility into rider safety status.

1.5 Purpose of the Project

The primary purpose of this project is to demonstrate the feasibility and effectiveness of an AI-powered smart helmet system through a working software prototype. The project aims to:

- Showcase the integration of simulated IoT sensor data with a Machine Learning prediction model.
- Provide a real-time web dashboard for monitoring rider safety status.
- Simulate emergency alert mechanisms including GPS location tracking and SMS dispatch.
- Serve as a foundation for future hardware implementation using actual sensors and microcontrollers.
- Demonstrate practical applications of AI, IoT, and web technologies in the domain of road safety.

1.6 Scope of the Project

The scope of this project encompasses a full-stack web application comprising a FastAPI backend (Python) and a React.js frontend (Vite), a Random Forest Classifier trained on synthetic helmet sensor data, software-based simulation of sensor readings, a professional web dashboard, simulated crash detection with GPS and SOS, and five distinct simulation modes for demonstration.

Out of scope for the current version: Physical hardware prototype with real sensors, actual GSM SMS transmission, mobile application development, cloud database integration, and real-world accident dataset training.

CHAPTER 2: PROBLEM DEFINITION

2.1 Existing Problem

Motorcycle accidents remain one of the most pressing road safety challenges worldwide. Despite advancements in vehicle technology and road network infrastructure, two-wheeler riders continue to face elevated risks due to their exposed riding position and the inherent instability of two-wheeled vehicles.

The existing approach to motorcycle safety relies primarily on passive helmets that only provide impact protection, manual accident reporting that depends on bystanders, and post-accident response where emergency services are activated only after an accident is reported. This reactive approach fails to address accident prevention and timely emergency response.

2.2 Limitations of Traditional Helmets

Traditional motorcycle helmets suffer from several significant limitations:

- No sensing capability: Traditional helmets cannot detect any parameters about the rider's condition.
- No communication capability: No built-in mechanism to communicate with external systems or emergency services.
- No predictive intelligence: Without sensors and processing, traditional helmets cannot anticipate dangers.
- No accident detection: Cannot automatically detect that an accident has occurred.
- No impairment detection: No capability to detect alcohol influence or dangerous fatigue.

2.3 Delayed Emergency Response Issue

One of the most critical factors in accident survival is the speed of emergency medical response. Medical research has established the concept of the "Golden Hour" - the critical time window (approximately 60 minutes) after a traumatic injury during which prompt medical treatment significantly improves survival rates.

In many motorcycle accident scenarios, especially those occurring in remote areas, during nighttime, single-vehicle accidents, or situations where the rider is unconscious, the delay between accident occurrence and emergency response arrival often exceeds the Golden Hour, dramatically reducing survival chances. An automated emergency alert system can significantly reduce this critical delay.

2.4 Alcohol, Drowsiness, and Risky Riding Problems

Riding under the influence of alcohol reduces reaction time, impairs judgment, compromises motor coordination, and induces overconfidence. Rider drowsiness leads to microsleep episodes, reduced situational awareness, and gradual loss of postural control. Overspeeding, aggressive acceleration, and sharp turns contribute to a large number of motorcycle accidents. Real-time monitoring of rider behavioral patterns helps identify at-risk conditions before accidents occur.

2.5 Problem Statement

Motorcycle riders face significant risks due to road accidents, unsafe riding behavior, delayed medical response, and lack of proactive safety systems. Traditional helmets provide physical protection but lack intelligent features that can predict accidents, monitor rider condition, or provide emergency communication.

There is a clear need for an intelligent helmet system that can continuously monitor rider parameters using multiple sensors, employ Artificial Intelligence to predict accident risk levels in real-time, automatically detect crash events and dispatch emergency alerts with GPS coordinates, detect and respond to alcohol impairment and rider drowsiness, and provide a real-time monitoring dashboard for safety oversight.

CHAPTER 3: OBJECTIVE OF THE PROJECT

The primary objectives of this project are as follows:

- To monitor rider safety using smart helmet sensor data: Design and implement a system that processes multiple sensor inputs - including speed, vibration, alcohol level, drowsiness score, 3-axis accelerometer, and 3-axis gyroscope data - to comprehensively assess the rider's condition.
- To classify rider condition as Safe, At-Risk, or Danger: Develop a Machine Learning model (Random Forest Classifier) capable of analyzing the 10-dimensional sensor feature vector and classifying the rider's condition into three risk levels with associated confidence probabilities.
- To detect risky riding, alcohol impairment, drowsiness, and crash conditions: Implement distinct detection mechanisms for overspeeding (Risky), elevated blood alcohol (Drunk), high fatigue index (Drowsy), and high-G impact (Crash).
- To provide a real-time web dashboard: Build a professional, responsive web dashboard using React.js that displays live sensor telemetry, AI predictions, historical charts, and emergency alert information.
- To simulate GPS-based emergency alert: Demonstrate an emergency SOS system that extracts GPS coordinates upon crash detection and generates a Google Maps link for emergency responders.
- To demonstrate AI and IoT integration through a software prototype: Create a working 50% software prototype that demonstrates the complete pipeline from sensor data collection to AI prediction to emergency response.
- To provide future scope for hardware implementation: Design the software architecture in a modular manner that allows straightforward integration with physical hardware components in future iterations.

CHAPTER 4: LITERATURE SURVEY

4.1 Smart Helmet Systems

The concept of smart helmets has evolved significantly in recent years with the advancement of embedded systems and IoT technologies. Modern approaches integrate multiple sensors, wireless communication modules, and intelligent processing capabilities. Research has explored the use of MEMS sensors within helmet cavities for real-time data acquisition. The integration of microcontrollers such as Arduino and ESP32 has enabled on-device processing of sensor data.

4.2 IoT-Based Accident Detection Systems

IoT technology has been extensively applied in accident detection and emergency response systems. Approaches typically involve sensor nodes for data collection, wireless communication protocols for data transmission, and cloud-based processing for analysis and alert generation. Key challenges include power management, reliable communication, and minimizing false positives.

4.3 Alcohol Detection in Vehicle Safety

Breath alcohol detection using semiconductor-based gas sensors such as the MQ-3 has been widely researched. Studies have demonstrated integration with vehicle ignition systems to create alcohol interlock mechanisms. The concept of integrating alcohol detection within a helmet is particularly relevant for two-wheelers.

4.4 Drowsiness Detection Systems

Drowsiness detection approaches include vision-based methods using cameras, physiological signal-based methods monitoring EEG/EOG/EMG, and behavioral pattern-based methods analyzing driving patterns. For helmet-based systems, IR sensors for eye blink detection combined with accelerometer/gyroscope behavioral analysis provides a practical approach.

4.5 GPS and GSM Emergency Alert Systems

The combination of GPS receivers (NEO-6M) and GSM modules (SIM800L) has been extensively used in vehicle tracking and emergency alert systems. Including a direct Google Maps URL with GPS coordinates in SMS messages significantly improves emergency response efficiency.

4.6 Machine Learning for Risk Prediction

Machine Learning algorithms including Decision Trees, Random Forests, SVMs, and Neural Networks have been applied to road safety applications. For this project, the Random Forest Classifier was selected due to its high accuracy on tabular data, robustness against overfitting, fast inference time, and interpretability through feature importance analysis.

4.7 Summary of Literature Survey

The literature survey reveals that smart helmet technology is an active research area combining IoT, AI, and communication technologies. Multi-sensor fusion, ML classifiers, GPS-GSM integration, and software prototyping before hardware development are proven approaches.

CHAPTER 5: REQUIREMENT ANALYSIS

5.1 Existing System

The existing system for motorcycle rider safety consists of conventional helmets that provide only passive physical protection. Helmets have no embedded sensors, accident detection relies on manual reporting, there is no real-time monitoring, and emergency response is delayed.

Table 1: Comparison of Existing and Proposed Systems

Feature	Existing System	Proposed System
Helmet Type	Passive (physical protection only)	Active (sensor-integrated)
Accident Detection	Manual (bystander reporting)	Automatic (sensor-based)
Risk Prediction	Not available	AI-based (Random Forest)
Alcohol Detection	Not available	MQ-3 sensor-based
Drowsiness Detection	Not available	IR sensor / fatigue score
Emergency Alert	Manual phone call	Automated GPS + GSM SMS
Real-time Monitoring	Not available	Web-based live dashboard
Response Time	15-60+ minutes	Immediate automated alert

5.2 Proposed System

The proposed system is an AI-Powered Smart Helmet that integrates multiple sensors, a Machine Learning prediction engine, and an automated emergency alert mechanism. The current implementation is a 50% working software prototype that simulates the complete system pipeline using FastAPI backend, Random Forest ML model, and React web dashboard.

5.3 Functional Requirements

Table 2: Functional Requirements

ID	Requirement	Description
FR-01	Display live sensor values	Real-time values for speed, alcohol, drowsiness, vibration, accelerometer, gyroscope
FR-02	Switch simulation modes	Switch between Safe, Risky, Drunk, Drowsy, Crash modes
FR-03	Predict rider state	Predicts rider condition: Safe, At-Risk, Danger
FR-04	Detect alcohol condition	Detect elevated alcohol and display warnings
FR-05	Detect drowsiness	Detect high drowsiness scores and display fatigue warnings
FR-06	Detect crash condition	Detect high-G impact and vibration spikes
FR-07	Display GPS location	Show GPS coordinates with Google Maps link
FR-08	Show emergency SOS	Display SOS card with crash details and GSM status

5.4 Non-Functional Requirements

Table 3: Non-Functional Requirements

ID	Requirement	Description
NFR-01	User-friendly interface	Intuitive, visually appealing dashboard
NFR-02	Fast response time	Updates within 1 second
NFR-03	Demo mode accuracy	Predictions consistent with selected mode
NFR-04	Scalability	Supports future hardware integration
NFR-05	Maintainability	Well-documented, modular codebase
NFR-06	Presentation readiness	Suitable for live demonstration

5.5 Software Requirements

Table 4: Software Requirements

Component	Technology	Details
Programming Language	Python	3.9+
Backend Framework	FastAPI	Latest stable
Frontend Framework	React.js	18.x with Vite 5.x
ML Library	Scikit-learn	Random Forest Classifier
Numerical Computing	NumPy, Pandas	Latest stable
Chart Library	Recharts	React charting
Icon Library	Lucide-React	SVG icons
IDE	VS Code / Antigravity IDE	Development environment

5.6 Hardware Requirements (Future Scope)

Table 5: Hardware Requirements (Future Scope)

Component	Specification	Purpose
ESP32	Dual-core, Wi-Fi + BLE	Central microcontroller
MPU6050	6-axis accel + gyro	Impact and orientation detection
MQ-3 Sensor	Semiconductor gas sensor	Breath alcohol detection
SW-420 Sensor	Vibration sensor	Mechanical impact detection
NEO-6M GPS	Satellite positioning	Real-time location tracking
SIM800L GSM	Quad-band GSM	Emergency SMS dispatch
Active Buzzer	Piezoelectric	Audible warning alerts
Relay Module	5V relay	Ignition lock control
Helmet	ISI-certified	Physical housing

Note: These hardware components are listed for future implementation. The current software prototype uses simulated sensor data.

CHAPTER 6: SYSTEM DESIGN

6.1 System Architecture

The system architecture follows a layered design pattern that separates concerns across hardware, communication, processing, and presentation layers. Layer 1 is the Sensor Hardware Layer (future scope) housing MPU6050, MQ-3, SW-420, NEO-6M, and IR sensors. Layer 2 is the ESP32 Microcontroller Layer. Layer 3 is the FastAPI Backend Processing Layer (implemented). Layer 4 is the AI/ML Prediction Layer with the Random Forest Classifier (implemented). Layer 5 is the React Frontend Presentation Layer (implemented). Layer 6 is the Emergency Alert Layer (simulated).

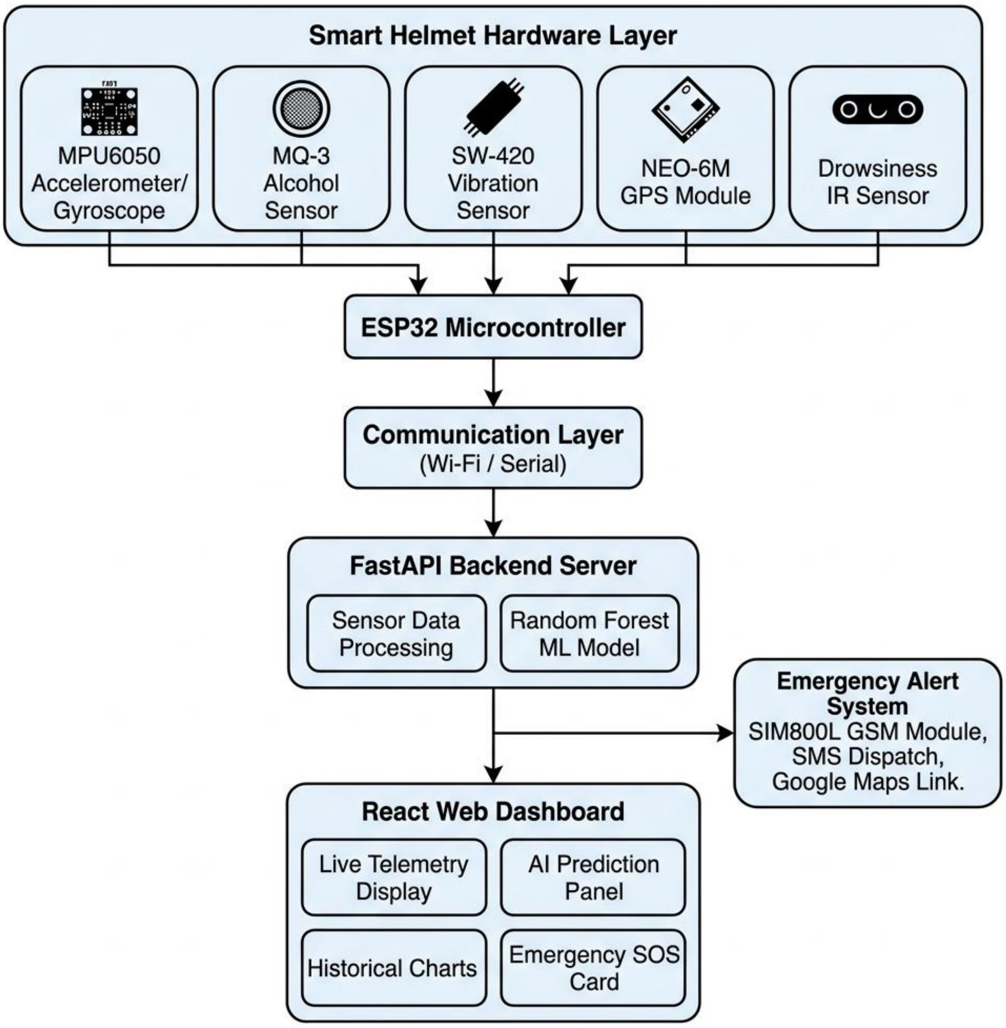

Fig. 1: System Architecture Diagram

6.2 Data Flow Diagram

The Level-1 Data Flow Diagram illustrates how data moves through the system. External entities include the Rider (generates sensor data), Emergency Contacts (receive SOS alerts), and Google Maps (location visualization). Key processes include Collect Sensor Data, Process and Predict Risk, Display Dashboard, and Trigger Emergency Alert. Data stores include the Sensor History Log and ML Model Store.

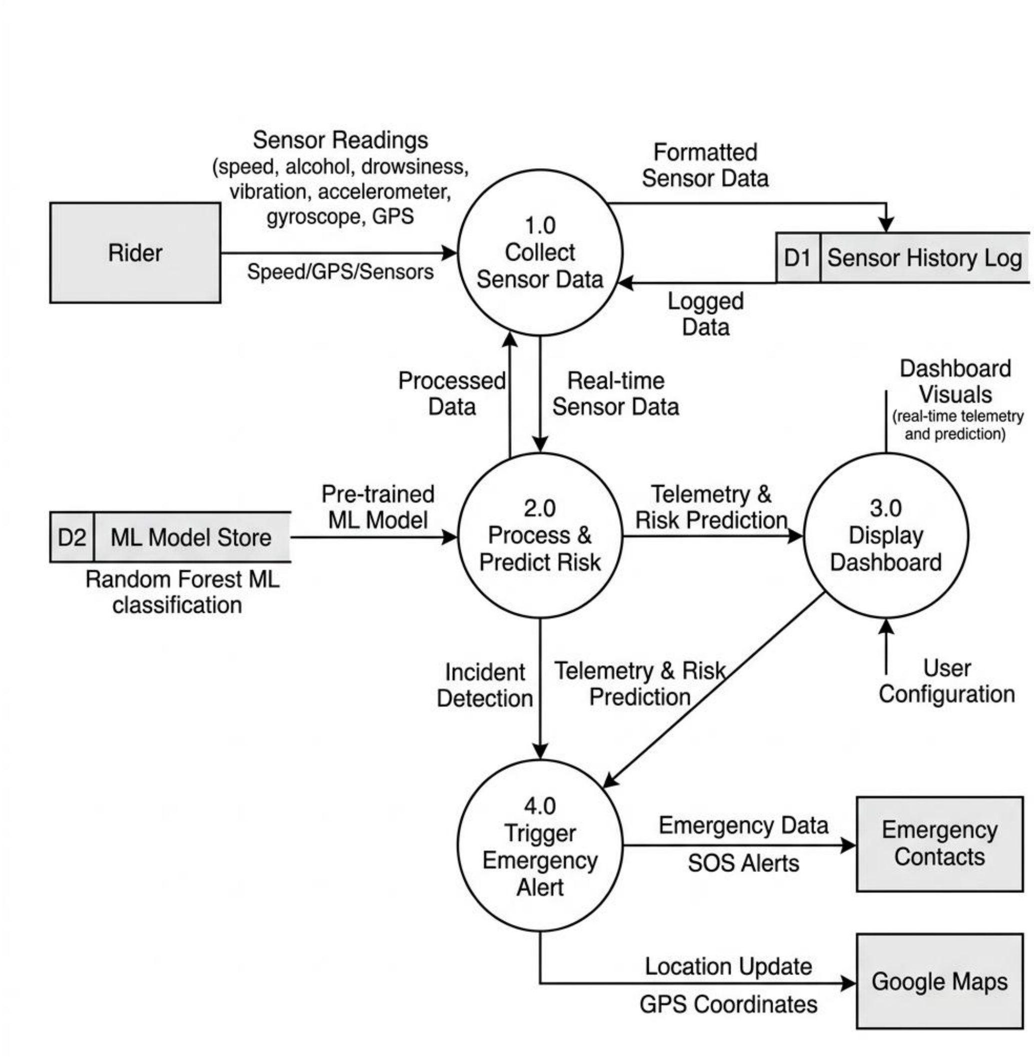

Fig. 2: Data Flow Diagram (Level-1)

6.3 Use Case Diagram

The Use Case Diagram identifies primary actors (Rider, Emergency Contact, System Administrator) and their interactions with the system including View Live Dashboard, Select Simulation Mode, View Sensor Telemetry, View AI Risk

Prediction, View GPS Location, Receive Emergency SOS Alert, View Historical Charts, Reset Alert, Train ML Model, and Configure Emergency Contacts.

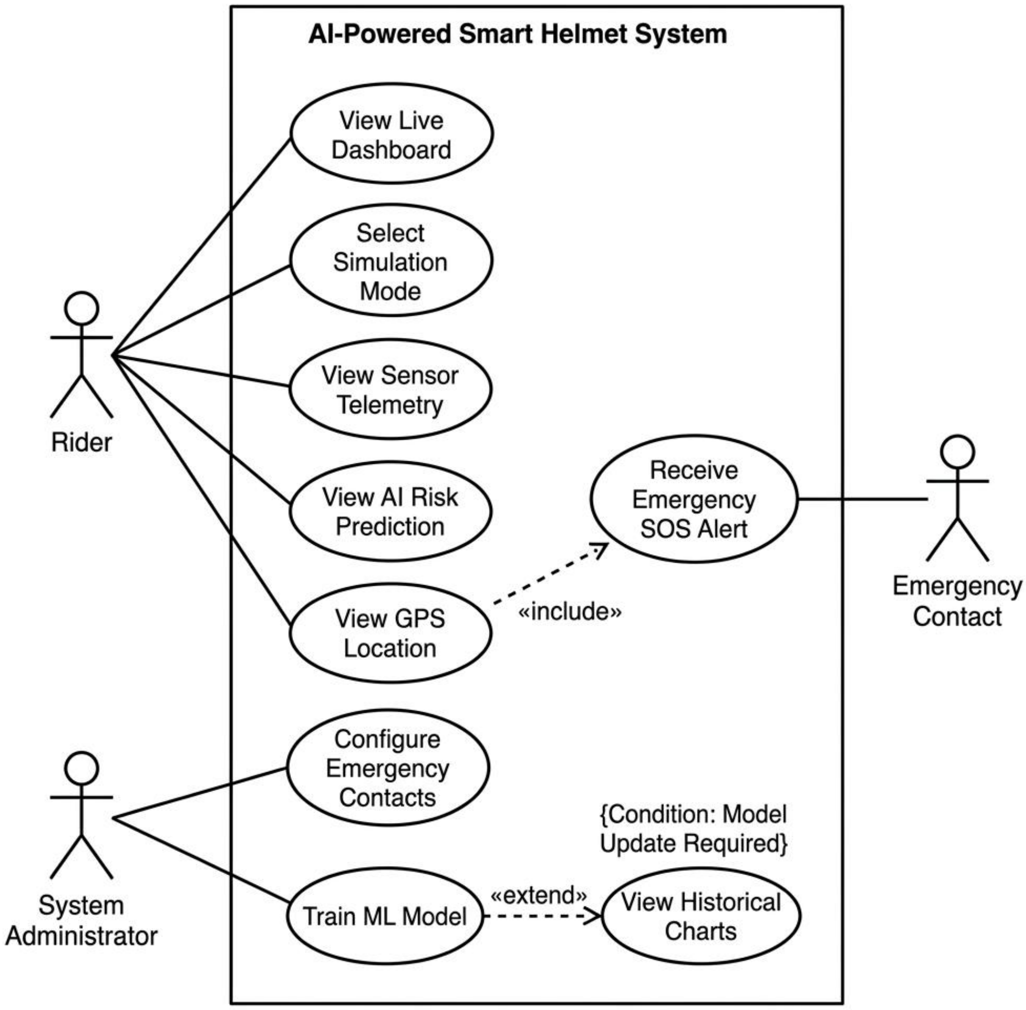

Fig. 3: Use Case Diagram

6.4 Activity Diagram

The activity diagram depicts the complete system workflow: from system initialization and sensor data collection to AI risk prediction, branching on risk state, emergency alert execution upon crash detection, GPS coordinate extraction, SOS SMS dispatch simulation, and real-time dashboard updates.

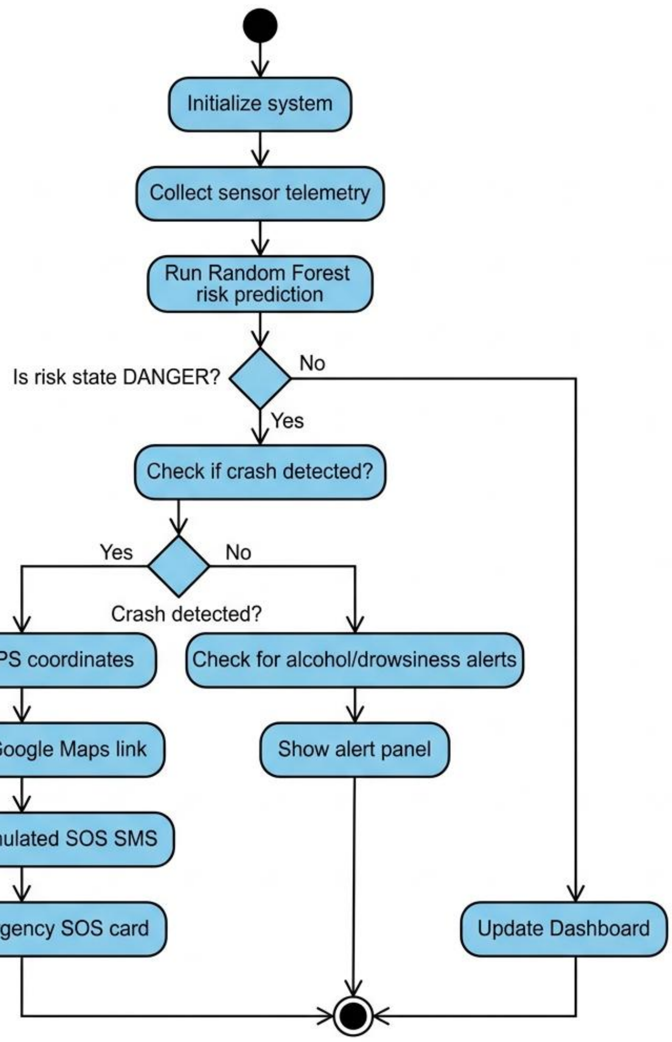

Fig. 4: Activity Diagram

6.5 Sequence Diagram

The sequence diagram shows the chronological interaction between the User, React Dashboard, FastAPI Backend, Random Forest Model, Sensor Simulation, and Emergency Alert System during mode selection, live data polling, and emergency SOS dispatch.

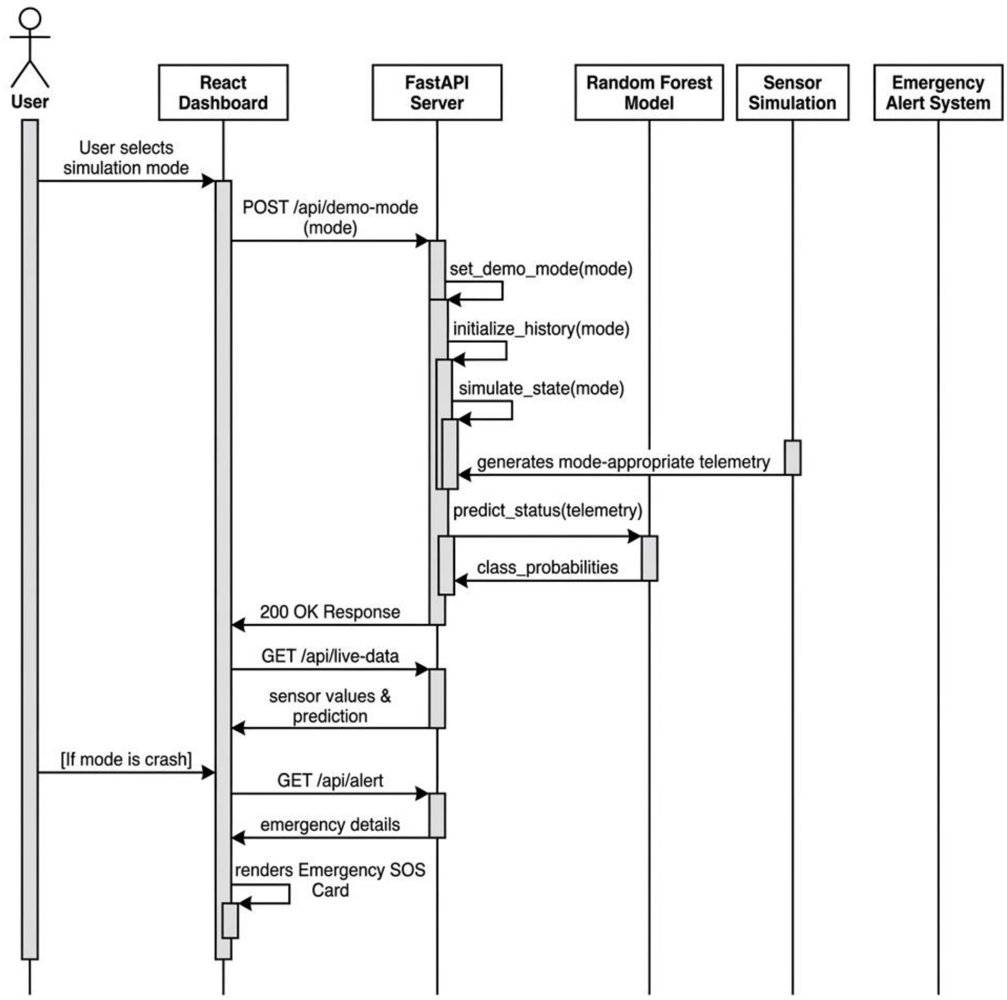

Fig. 5: Sequence Diagram

6.6 Class Diagram

The UML Class Diagram illustrates the object-oriented structure of the backend and frontend modules, detailing class attributes, methods, and relationships between HelmetState, DemoModeRequest, FastAPI App, RandomForestClassifier, and ReactDashboard.

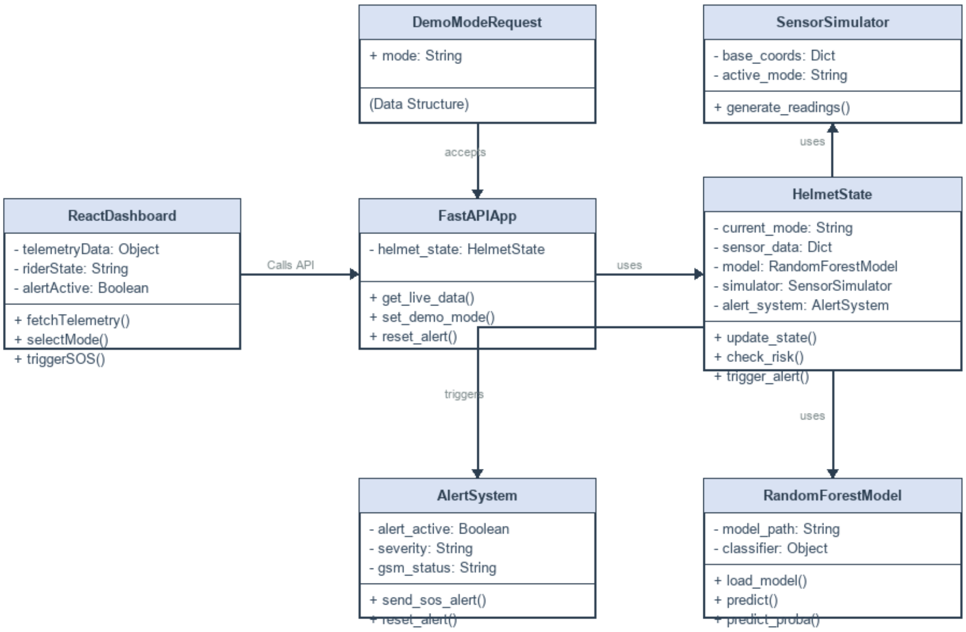

Fig. 6: Class Diagram

6.7 Architecture Overview

The complete architecture diagram (Fig. 1) illustrates the end-to-end system showing data flow from the sensor hardware layer through communication and processing layers to the presentation and alert layers. The modular design ensures the simulation layer can be replaced with real hardware interfaces in future iterations without modifying core prediction and presentation logic.

CHAPTER 7: IMPLEMENTATION

7.1 Implementation Overview

The implementation follows a modular full-stack architecture with clear separation between the backend processing layer and the frontend presentation layer. The backend directory contains the FastAPI server (main.py), ML model module (model.py), trained model file (helmet_model.joblib), and dependency specifications. The frontend directory contains the React application with Vite, page components, API layer, and styling.

7.2 Backend Implementation

The backend is built using Python FastAPI. It provides four REST API endpoints: GET /api/live-data returns current sensor telemetry, ML predictions, and historical readings. GET /api/alert returns the emergency alert status. POST demo mode endpoint switches the simulation to a specified mode. POST /api/reset-alert resets the alert state and returns to safe mode.

State management uses a singleton HelmetState object with a threading.Lock for thread-safe access. A daemon background thread runs continuously, calling simulate_state() every second to update sensor values based on the active demo mode. CORS middleware is configured for frontend access.

7.3 Frontend Implementation

The frontend is a React.js single-page application built with Vite. The Home page displays the project overview, problem definition, team section with role-based icons and contribution badges, and a future hardware integration diagram. The Live Dashboard features mode selector buttons, an AI prediction panel, six sensor telemetry cards, two historical trend charts (Recharts), and an emergency SOS card. The Modules page describes all eight project modules.

7.4 Machine Learning Implementation

The system uses a Random Forest Classifier from scikit-learn. The model is trained on 1,500 synthetically generated samples with 10 features (speed, vibration, alcohol_level, drowsy_score, accel_x, accel_y, accel_z, gyro_x, gyro_y, gyro_z) and

3 class labels (Safe, At-Risk, Danger). The model uses 80 decision trees with maximum depth of 8. Labels are assigned using rule-based logic based on sensor value thresholds for each hazard type.

Table 6: ML Prediction Labels and Probabilities

Demo Mode	Predicted Class	Safe %	At-Risk %	Danger %
Safe	Safe	92	8	0
Risky	At-Risk	5	92	3
Drunk	Danger	0	5	95
Drowsy	Danger	0	12	88
Crash	Danger	0	0	100

In demo mode, predictions are deterministically aligned with the selected mode to ensure consistent demonstration behavior without contradictions.

7.5 Sensor Simulation Implementation

Each simulation mode generates sensor values within realistic ranges. The speedometer ranges from 30-50 km/h in safe mode to 85-95 km/h in risky mode. The alcohol sensor ranges from 0.01-0.05 mg/L (safe) to 0.38-0.45 mg/L (drunk). The drowsiness tracker ranges from 0-15% (safe) to 82-94% (drowsy). Mechanical vibration ranges from 0.4-1.2 m/s² (safe) to 18.7 m/s² (crash impact). GPS coordinates simulate slow drift with coordinates frozen during crash events.

7.6 Simulation Modes

Table 7: Simulation Mode Parameters

Mode	Speed	Alcohol	Drowsiness	Vibration	Prediction
Safe	30-50 km/h	0.01-0.05	0-15%	0.4-1.2 m/s²	Safe
Risky	85-95 km/h	0.04-0.11	15-28%	1.8-3.5 m/s²	At-Risk
Drunk	30-42 km/h	0.38-0.45	8-25%	0.8-2.2 m/s²	Danger
Drowsy	52-60 km/h	0.00-0.04	82-94%	0.6-1.6 m/s²	Danger
Crash	72 to 0 km/h	0.02	5%	18.7 m/s²	Danger

7.7 Emergency Alert Implementation

The emergency alert system activates when a crash event is detected. The crash sequence progresses over multiple ticks: Tick 0 shows normal riding at 72.4 km/h. Tick 1 simulates the impact event with speed dropping to 18.2 km/h, vibration spiking to 18.7 m/s^2, and accelerometer registering 5.2 G horizontal force. Tick 2+ shows the vehicle stopped at 0 km/h with impact readings persisting and the alert activated.

Upon crash detection, the system extracts GPS coordinates, generates a Google Maps URL, simulates SIM800L GSM SMS dispatch with crash details, and displays an emergency SOS card on the dashboard with severity, impact force, GPS coordinates, and GSM status.

7.8 Ignition Lock Explanation

The ignition lock feature prevents intoxicated riding by disabling the motorcycle starting system when unsafe alcohol levels are detected. When the MQ-3 sensor detects BAC above 0.35 mg/L, a relay module disconnects the ignition circuit. Important safety note: The lock prevents starting the motorcycle - it should never cut the engine of a moving motorcycle. In the current software prototype, this is simulated through the prediction display.

CHAPTER 8: MODULES DESCRIPTION

8.1 Sensor Monitoring System

The foundational module managing collection and preprocessing of all sensor data. Generates 10-dimensional sensor readings at 1-second intervals, maintains sensor value ranges appropriate to the selected simulation mode, provides GPS coordinate tracking with simulated drift, and manages transitions between sensor value profiles. Simulated sensors include MPU6050 Accelerometer/Gyroscope (6-axis), MQ-3 Alcohol Sensor, SW-420 Vibration Sensor, Speed Sensor, and IR Drowsiness Sensor.

8.2 AI/ML Risk Prediction Module

The intelligence core analyzing sensor data and classifying rider condition using a trained Random Forest Classifier with 80 decision trees and maximum depth of 8. Processes a 10-dimensional feature vector and outputs Safe, At-Risk, or Danger classification with per-class probability distributions. The trained model is serialized using joblib.

8.3 Alcohol Detection Module

Monitors blood alcohol concentration using MQ-3 sensor simulation. Detection thresholds: Below 0.15 mg/L is Safe, 0.15-0.35 mg/L is At-Risk, above 0.35 mg/L is Danger. Response actions include dashboard warnings, Danger prediction status, and future ignition lock relay.

8.4 Drowsiness Detection Module

Monitors rider fatigue level using eye blink pattern analysis simulation. The drowsiness score ranges from 0% to 100%. Below 50% is Safe, 50-75% is At-Risk, above 75% is Danger. Future hardware implementation includes a high-pitch buzzer alarm inside the helmet.

8.5 Crash and Impact Detection Module

Analyzes accelerometer and vibration data to detect crash events. Detection criteria include vibration exceeding 12.0 m/s^2, accelerometer exceeding 4.0 G, or combined speed above 40 km/h with vibration above 8.0 m/s^2. The crash sequence simulates pre-crash, impact, and post-crash phases with automatic emergency alert activation.

8.6 GPS Location Tracking Module

Provides real-time geographic coordinates. Currently simulated with base coordinates at Bengaluru, India (12.9716°N, 77.5946°E) with slow drift for movement simulation. Coordinates are frozen upon crash detection. Generates direct Google Maps URLs for location visualization.

8.7 GSM Emergency SMS Alert Module

Handles dispatch of emergency notifications upon crash detection. Currently simulated with SMS content generation including crash severity, impact force, GPS coordinates, Google Maps URL, and timestamp. Future implementation uses SIM800L GSM module with AT command protocol.

8.8 React Live Dashboard Module

Primary user interface for real-time monitoring. Components include mode selector panel, AI prediction card, six sensor telemetry cards, two historical trend charts (Recharts), emergency SOS card, and GPS coordinates display. Polls backend every second for fresh data.

8.9 Data Logging and Charts Module

Manages historical sensor data storage and visualization. Maintains a rolling window of 20 sensor readings with timestamps. Pre-populates history with realistic trends when mode changes. Renders speed and vibration area charts with smooth animations.

8.10 Future Hardware Integration Module

Outlines the planned architecture for transitioning to physical hardware. Pipeline: Helmet Sensors -> ESP32 (I2C, Analog, Digital) -> Wi-Fi -> FastAPI Backend -> ML Prediction -> React Dashboard. NEO-6M GPS via UART, SIM800L GSM via UART, Active Buzzer via GPIO, Relay Module via GPIO for ignition lock.

CHAPTER 9: OUTPUT AND SCREENSHOTS

This chapter presents the output screenshots captured from the AI-Powered Smart Helmet web application, demonstrating the various features, interface views, and simulation modes.

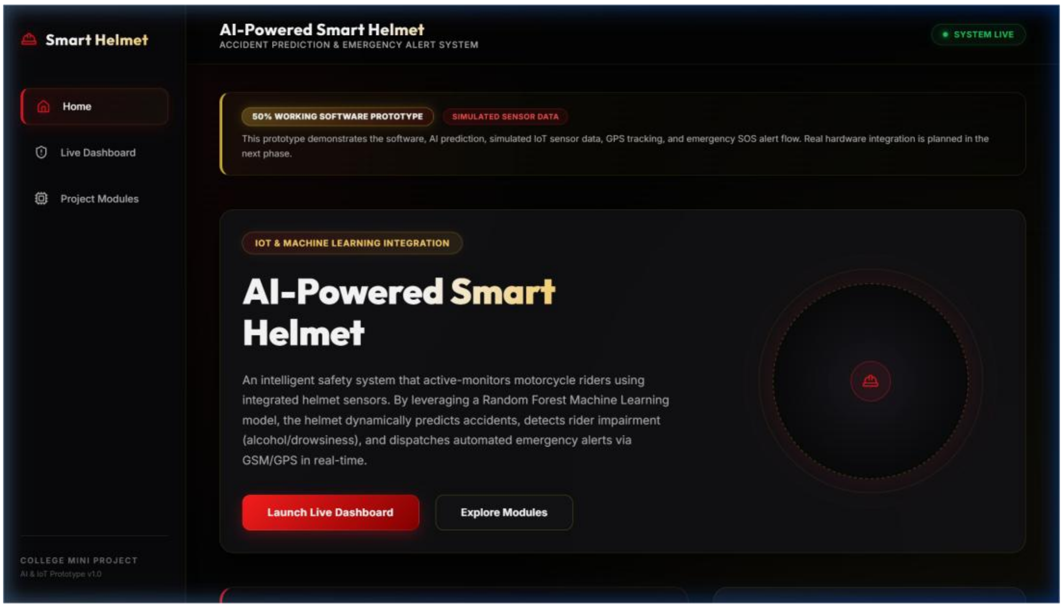

Fig. 7: Home Page of Smart Helmet Website

The Home page displays the project title, abstract, navigation buttons, problem definition, monitoring modules overview, project highlights, team details, and future hardware integration information. The design uses a professional dark UI design with red-gold accent colors.

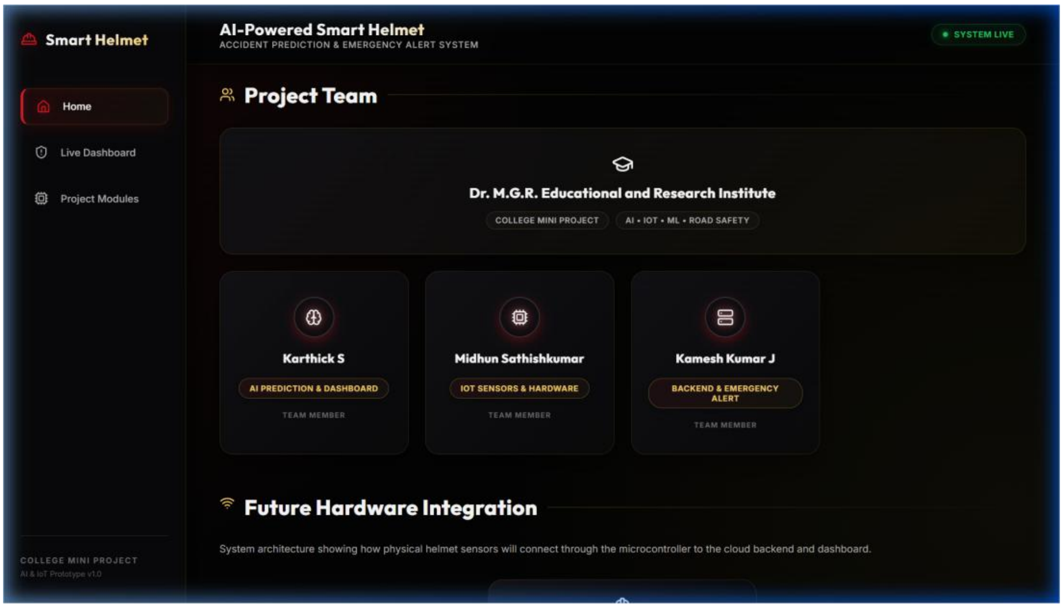

Fig. 8: Project Team Section

Team member cards with role-based icons (Brain for AI, CPU for IoT, Server for Backend), member names, and contribution badges. College name displayed prominently.

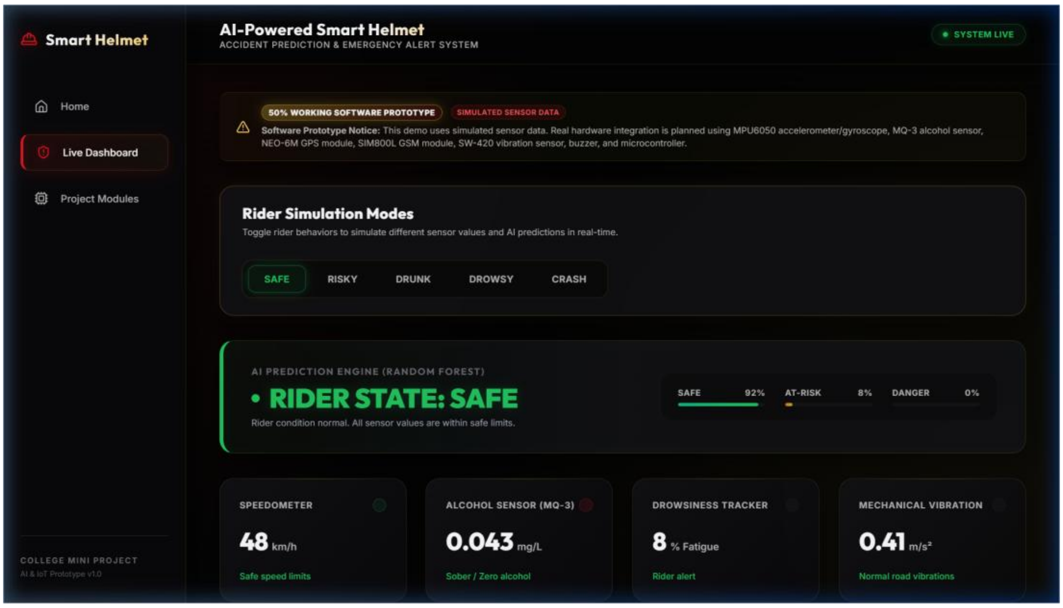

Fig. 9: Live Dashboard in Safe Mode

All sensor values within normal ranges. The AI prediction panel clearly shows RIDER STATE: SAFE with green styling, speed between 30-50 km/h, negligible alcohol, drowsiness below 15%.

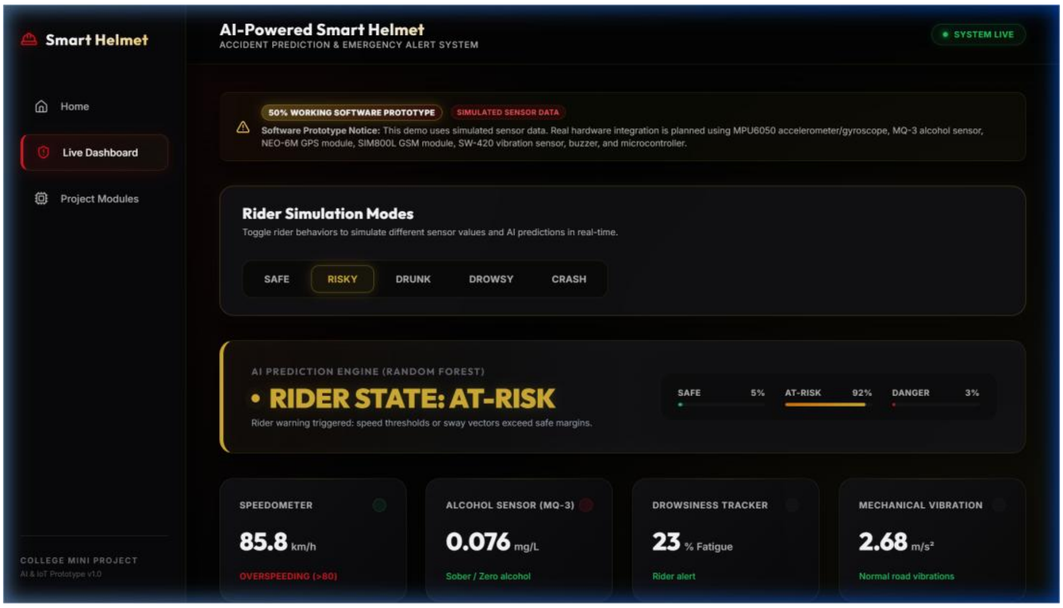

Fig. 10: Risky Mode Showing At-Risk Prediction

Elevated speed (85-95 km/h). The AI prediction panel clearly shows RIDER STATE: AT-RISK with amber styling, sensor values indicate overspeeding behavior.

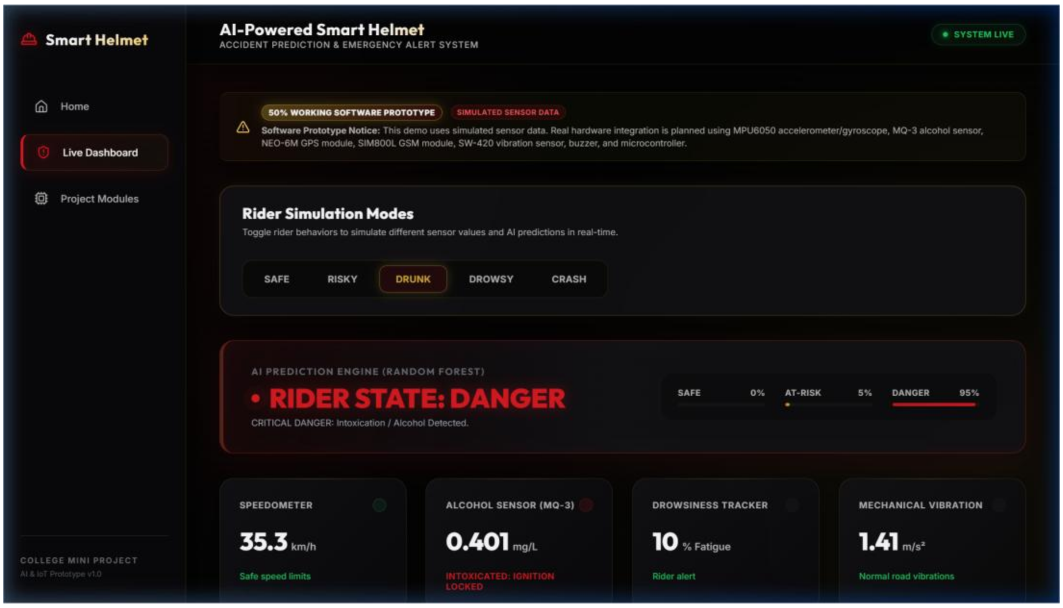

Fig. 11: Drunk Mode Showing Alcohol Detection

Alcohol sensor shows elevated readings (0.38-0.45 mg/L). The AI prediction panel clearly shows RIDER STATE: DANGER with red styling, alcohol detection warning message displayed.

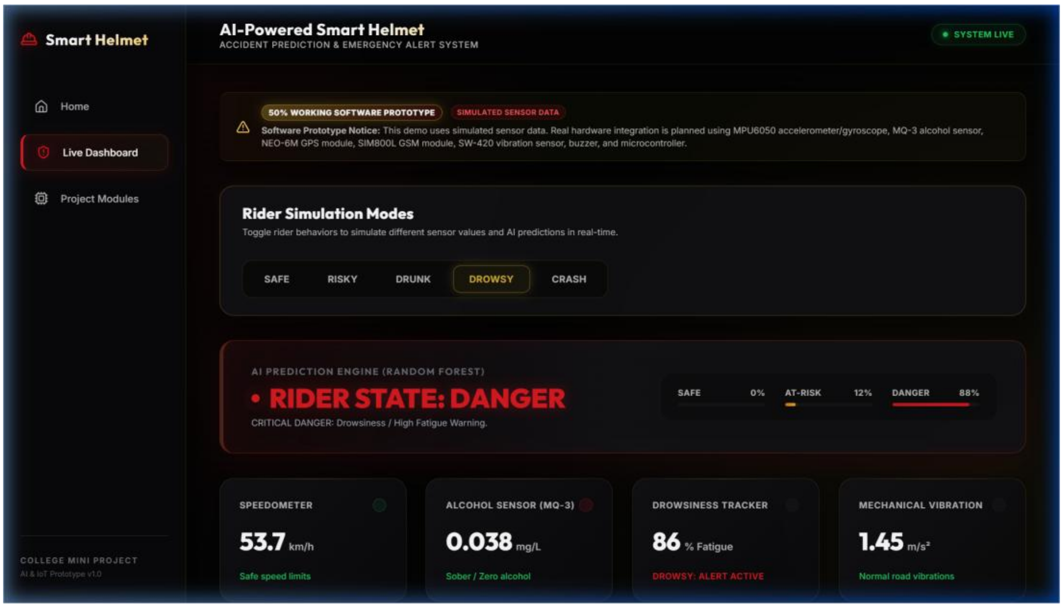

Fig. 12: Drowsy Mode Showing Fatigue Warning

Drowsiness tracker shows high fatigue scores (82-94%). The AI prediction panel clearly shows RIDER STATE: DANGER with red styling, fatigue/drowsiness warning message displayed.

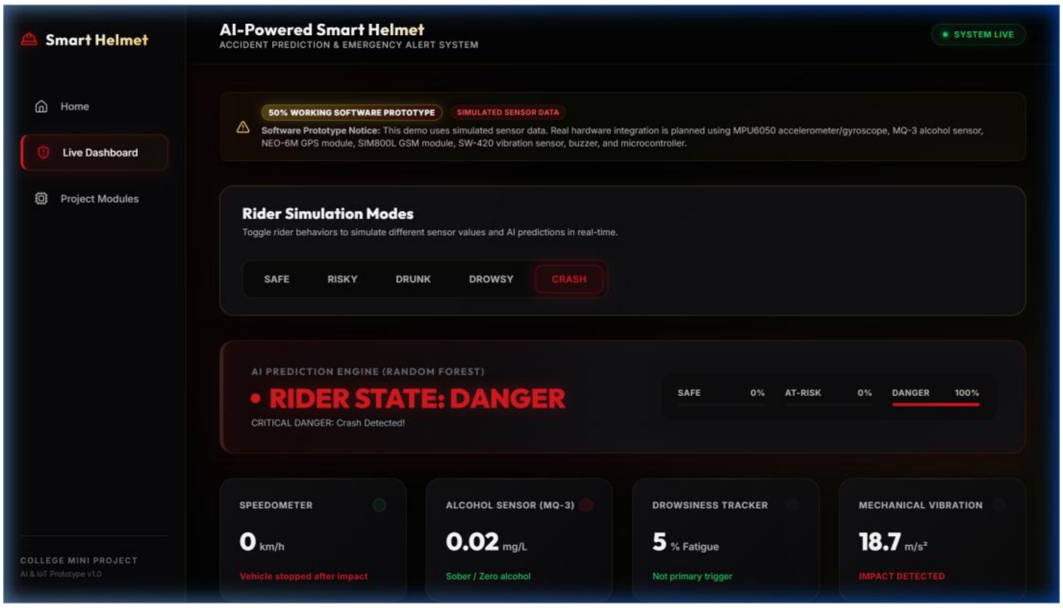

Fig. 13: Crash Mode Showing Emergency SOS Alert

The AI prediction panel clearly shows RIDER STATE: DANGER with red styling. Emergency SOS Alert card displayed prominently with crash severity (Critical), impact force (5.2 G / 18.7 m/s²), GPS coordinates, Google Maps link, and GSM status (Sent).

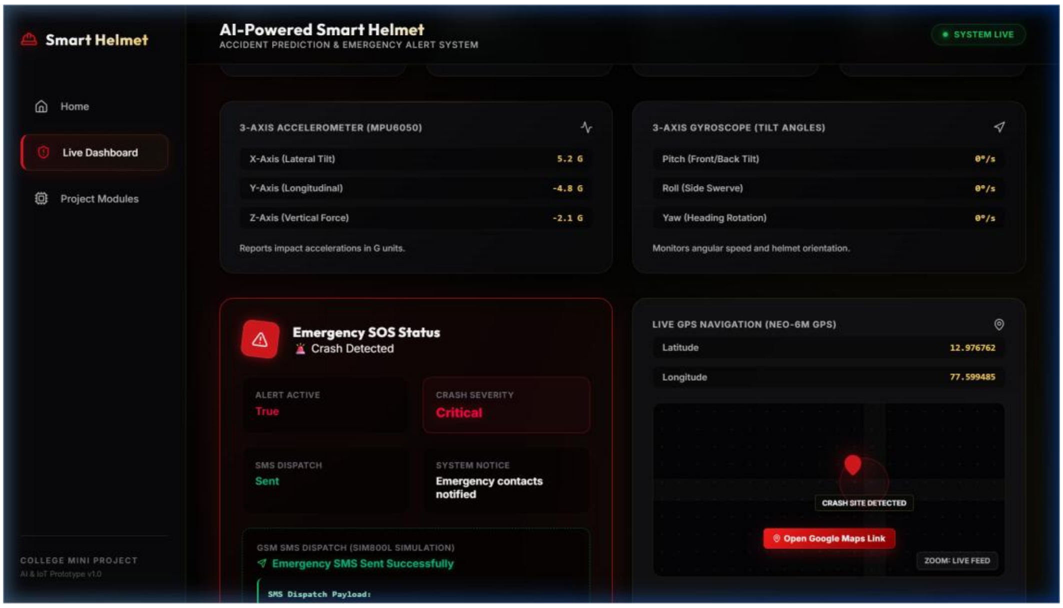

Fig. 14: GPS Location and Google Maps Link

GPS section shows rider's latitude and longitude with a direct clickable Google Maps link.

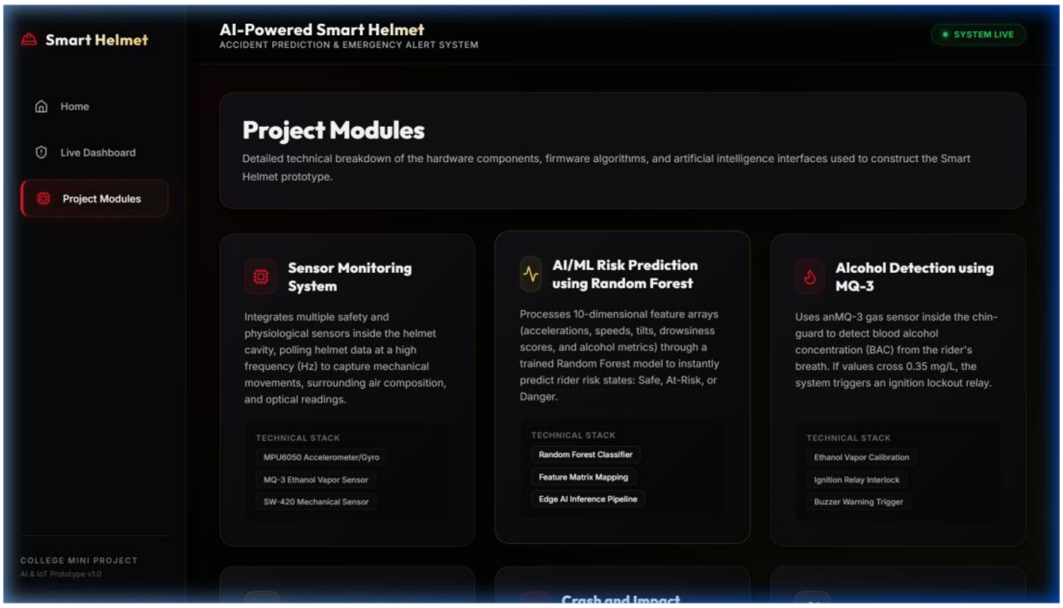

Fig. 15: Project Modules Page

All eight project modules displayed with icons, descriptions, and technical specifications.

Fig. 16: Future Hardware Integration Diagram

Visual pipeline diagram showing the planned sensor-to-cloud architecture.

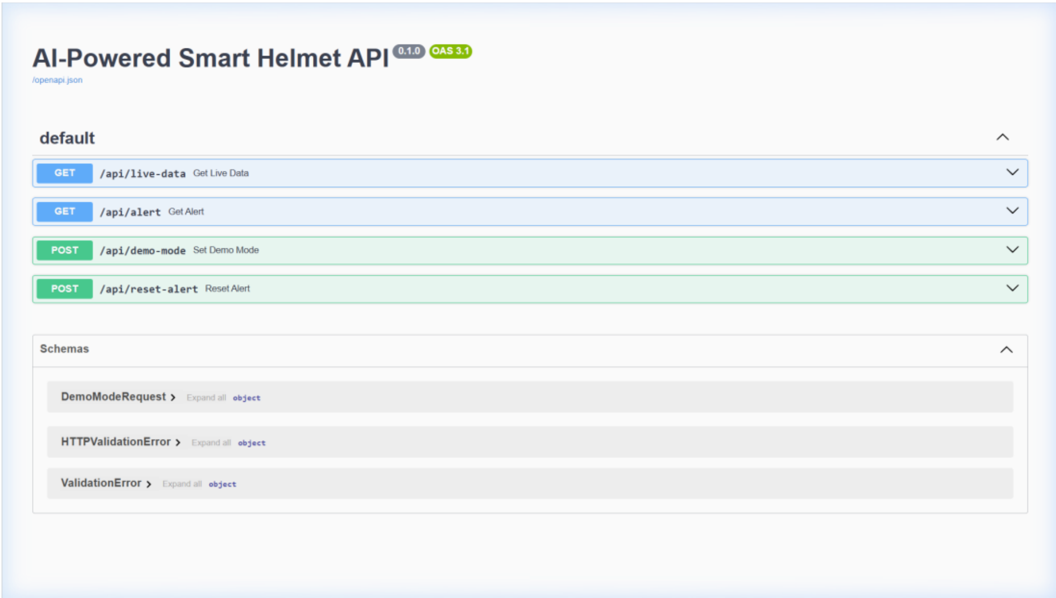

Fig. 17: FastAPI Documentation Page

The FastAPI interactive documentation (Swagger UI) displaying the REST API endpoints and data schemas.

CHAPTER 10: TESTING

10.1 Testing Overview

The system was tested comprehensively to ensure correct functionality across all simulation modes, API endpoints, and dashboard components. Testing was performed using manual functional testing, API endpoint verification, and UI behavior validation.

10.2 Test Cases and Results

Table 8: Test Cases and Results

TC ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC-01	Safe mode prediction	Select Safe	SAFE, Green, 30-50 km/h	SAFE, Green, 30-50 km/h	Pass
TC-02	Risky mode prediction	Select Risky	AT-RISK, Amber, 85-95 km/h	AT-RISK, Amber, 85-95 km/h	Pass
TC-03	Drunk mode prediction	Select Drunk	DANGER, Red, Alcohol 0.38+	DANGER, Red, Alcohol 0.38+	Pass
TC-04	Drowsy mode prediction	Select Drowsy	DANGER, Red, Drowsy 82%+	DANGER, Red, Drowsy 82%+	Pass
TC-05	Crash mode prediction	Select Crash	DANGER, SOS, Vibration 18.7	DANGER, SOS, Vibration 18.7	Pass
TC-06	Emergency SOS	Crash after impact	SOS card, GPS, GSM Sent	SOS card, GPS, GSM Sent	Pass
TC-07	GPS link	Click Maps link	Opens Google Maps	Opens Google Maps	Pass
TC-08	API live-data	GET /api/live-data	JSON with sensor data	JSON with sensor data	Pass
TC-09	API alert	GET /api/alert	Alert details JSON	Alert details JSON	Pass

TC ID	Test Scenario	Input	Expected Output	Actual Output	Status
TC-10	API mode switch	POST demo mode endpoint	200 OK, mode updated	200 OK, mode updated	Pass
TC-11	Dashboard render	Open dashboard	All cards render	All cards render	Pass
TC-12	Mode switch speed	Rapid switch	Immediate update	Immediate update	Pass

10.3 Testing Summary

All 12 test cases passed successfully. The system demonstrates consistent behavior across all simulation modes with predictions accurately matching the selected mode. API endpoints respond correctly and the dashboard renders all components without errors.

CHAPTER 11: ADVANTAGES AND APPLICATIONS

11.1 Advantages

- Improved Rider Safety: Continuous real-time monitoring of multiple rider parameters enables proactive identification of dangerous conditions.
- Reduced Emergency Response Delay: Automatic crash detection and GPS-based SOS alerts eliminate dependency on bystander reporting.
- Alcohol and Drowsiness Detection: Addresses two major causes of motorcycle accidents with real-time warnings.
- AI-Based Risk Prediction: Random Forest ML model provides nuanced risk assessment with confidence probabilities.
- GPS-Based Emergency Location: Direct Google Maps links ensure emergency responders can navigate to exact accident locations.
- Useful for Smart Mobility Systems: Scalable architecture supports fleet management and centralized safety monitoring.

11.2 Applications

- Motorcycle rider safety for daily commutes and long-distance travel.
- Delivery rider safety for food delivery and courier companies.
- Industrial helmet safety for construction, mining, and manufacturing environments.
- Fleet monitoring and management for transportation companies.
- Road safety research for studying riding patterns and accident causes.
- College IoT/AI project demonstration for academic purposes.

CHAPTER 12: LIMITATIONS

- Current version uses simulated sensor data rather than real hardware sensors.
- Real sensors (MPU6050, MQ-3, SW-420, NEO-6M) are not yet connected; hardware integration is planned as future scope.
- SMS alert is simulated; no actual GSM SMS transmission through cellular network.
- ML model trained on synthetic data; real-world accuracy may differ.
- Accuracy depends on real sensor quality, calibration, and reliability in future hardware implementation.
- Physical sensors require careful calibration for each specific helmet and riding environment.
- Internet/network dependency for communication between sensor module and backend server.
- Current system monitors a single rider; multi-rider fleet monitoring requires architectural modifications.
- No dedicated mobile application; dashboard is web-based only.
- Power consumption management needed for battery-operated hardware in future implementation.

CHAPTER 13: FUTURE SCOPE

• Real helmet hardware integration with embedded sensors in the helmet cavity.

• ESP32/Arduino integration with firmware for sensor collection and Wi-Fi communication.

• Real-time GPS tracking using NEO-6M GPS module with satellite-based coordinates.

• Actual GSM SMS alert dispatch using SIM800L module with cellular network.

• Cloud database integration (Firebase, AWS, GCP) for persistent data logging.

• Mobile application development for iOS and Android with push notifications.

• Real accident dataset training for improved ML model accuracy.

• Hospital and emergency service integration for automated emergency dispatch.

• Ignition lock using relay module to prevent intoxicated riding.

• Voice alert and buzzer system inside the helmet for immediate rider feedback.

• Helmet wearing detection using pressure or proximity sensors.

• Battery-powered compact PCB design for all electronic components.

CHAPTER 14: CONCLUSION

The AI-Powered Smart Helmet with Accident Prediction and Emergency Alert System successfully demonstrates the integration of Artificial Intelligence, Internet of Things, and modern web technologies to address the critical problem of motorcycle rider safety.

Through this mini project, we have developed a 50% working software prototype that showcases the complete data pipeline - from sensor data acquisition (simulated) to AI-based risk prediction using a Random Forest Classifier, and finally to emergency alert dispatch simulation with GPS coordinates.

The key achievements include: A Random Forest Classifier trained on a 10-dimensional sensor feature space capable of classifying rider conditions into three risk levels. A professional-grade full-stack web application with FastAPI backend and React.js frontend. Five distinct simulation modes demonstrating the system response to different rider conditions. An automated crash detection and emergency response system with GPS and Google Maps integration. A modular architecture designed for seamless transition to physical hardware.

The project validates the concept that an intelligent helmet system can significantly improve motorcycle rider safety by providing continuous monitoring, proactive risk prediction, and automated emergency response. The software prototype serves as a solid foundation for future hardware development using ESP32 microcontrollers and the planned sensor suite.

This project has provided valuable hands-on experience in full-stack web development, Machine Learning model training and deployment, REST API design, real-time telemetry visualization, and system architecture design - skills directly applicable to professional software engineering and IoT development careers.

CHAPTER 15: REFERENCES / BIBLIOGRAPHY

[1] FastAPI - Modern, Fast Web Framework for Building APIs with Python 3.7+. Official Documentation. https://fastapi.tiangolo.com/

[2] React - A JavaScript Library for Building User Interfaces. Official Documentation. https://react.dev/

[3] Vite - Next Generation Frontend Tooling. Official Documentation. https://vitejs.dev/

[4] Scikit-learn - Machine Learning in Python. Official Documentation. https://scikit-learn.org/

[5] NumPy - The Fundamental Package for Scientific Computing with Python. https://numpy.org/

[6] Pandas - Python Data Analysis Library. https://pandas.pydata.org/

[7] Recharts - A Composable Charting Library Built on React Components. https://recharts.org/

[8] World Health Organization (WHO), "Global Status Report on Road Safety," 2023.

[9] National Crime Records Bureau (NCRB), Ministry of Home Affairs, Government of India, "Accidental Deaths and Suicides in India," Annual Report.

[10] InvenSense Inc., "MPU-6050 Six-Axis MEMS MotionTracking Device," Product Datasheet.

[11] Zhengzhou Winsen Electronics, "MQ-3 Semiconductor Sensor for Alcohol Detection," Technical Datasheet.

[12] U-blox AG, "NEO-6M GPS Module," Product Datasheet and Integration Manual.

[13] SIMCom Wireless Solutions, "SIM800L GSM/GPRS Module," Hardware Design Guide.

[14] Espressif Systems, ESP32 Technical Reference Manual. https://www.espressif.com/

[15] General references on IoT-based accident detection, smart helmets, drowsiness detection, and alcohol detection.

APPENDIX A: BACKEND API DETAILS

API Endpoints Summary

Method	Endpoint	Description
GET	/api/live-data	Returns sensor data, prediction, history, mode
GET	/api/alert	Returns emergency alert status and details
POST	/api/demo-mode	Switches simulation mode
POST	/api/reset-alert	Resets alert, returns to safe mode

A.1 GET /api/live-data Response Example


```
{
  "demo_mode": "safe",
  "sensor_data": {
    "speed": 42.3,
    "vibration": 0.87,
    "alcohol_level": 0.023,
    "drowsy_score": 0.082,
    "accel_x": 0.05, "accel_y": -0.03, "accel_z": 1.02,
    "gyro_x": 1.4, "gyro_y": -0.8, "gyro_z": 2.1,
    "latitude": 12.9716, "longitude": 77.5946
  },
  "prediction": {
    "status": "Safe",
    "class_probabilities": {"Safe": 0.92, "At-Risk": 0.08, "Danger": 0.0}
  },
  "history": [...],
  "alert_active": false
}
```


A.2 GET /api/alert Response Example (Crash Active)


```
{
  "is_active": true,
  "severity": "Critical",
  "impact_force": "5.2 G (Horizontal) / 18.7 Vibration",
  "timestamp": "2025-01-15 14:32:18",
  "latitude": 12.9716, "longitude": 77.5946,
  "gsm_status": "Sent"
}
```

APPENDIX B: IMPORTANT CODE SNIPPETS

B.1 Random Forest Model Training (model.py)

```
def train_model():
    df = generate_synthetic_data(1500)
    X = df.drop(columns=["label"])
    y = df["label"]
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.2, random_state=42
    )
    model = RandomForestClassifier(
        n_estimators=80, max_depth=8, random_state=42
    )
    model.fit(X_train, y_train)
    joblib.dump(model, MODEL_PATH)
    return model
```

B.2 Prediction Function (model.py)

```
def predict_status(sensor_data: dict) -> dict:
    feature_names = [
        "speed", "vibration", "alcohol_level", "drowsy_score",
        "accel_x", "accel_y", "accel_z",
        "gyro_x", "gyro_y", "gyro_z"
    ]
    features = [float(sensor_data.get(name, 0.0))
                for name in feature_names]
    features_arr = np.array([features])
    prediction = model.predict(features_arr)[0]
    probabilities = model.predict_proba(features_arr)[0]
    status_map = {0: "Safe", 1: "At-Risk", 2: "Danger"}
    return {
        "status": status_map[prediction],
        "class_probabilities": {
            "Safe": float(probabilities[0]),
            "At-Risk": float(probabilities[1]),
            "Danger": float(probabilities[2])
        }
    }
```

B.3 FastAPI Live Data Endpoint (main.py)

```
@app.get("/api/live-data")
def get_live_data():
    with state.lock:
        return {
            "demo_mode": state.demo_mode,
            "sensor_data": state.sensor_data,
            "prediction": state.prediction,
            "history": state.history,
            "alert_active": state.alert_active
        }
```

B.4 Frontend API Layer (api.js)

```
const BASE_URL = 'http://127.0.0.1:8080';

export async function fetchLiveData() {
    const res = await fetch(`${BASE_URL}/api/live-data`);
    return res.json();
}

export async function setDemoMode(mode) {
    const res = await fetch(`${BASE_URL}/api/demo-mode`, {
        method: 'POST',
        headers: { 'Content-Type': 'application/json' },
        body: JSON.stringify({ mode })
    });
    return res.json();
}
```

APPENDIX C: RUN COMMANDS

C.1 Backend Setup and Run

```
cd backend
python -m venv .venv
.venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --reload --port 8080
```

Backend API documentation (Swagger UI) available at: http://127.0.0.1:8080/docs

C.2 Frontend Setup and Run

```
cd frontend
npm install
npm run dev
```

Frontend application available at: http://localhost:5174/

APPENDIX D: DEMO FLOW

1. Open Home Page - Explain the project title, abstract, and problem definition.

2. Scroll to Project Team - Show team members, contribution badges, and college affiliation.

3. Scroll to Hardware Integration - Walk through the planned sensor-to-cloud architecture.

4. Navigate to Live Dashboard - Point out the prototype badge and simulation disclaimer.

5. Start in Safe Mode - Show green SAFE prediction with stable sensor values.

6. Switch to Risky Mode - Highlight amber AT-RISK state with elevated speed.

7. Switch to Drunk Mode - Show DANGER state with alcohol detection warning.

8. Switch to Drowsy Mode - Demonstrate fatigue warning and drowsiness alerts.

9. Switch to Crash Mode - Showcase emergency SOS card, GPS, Google Maps link, GSM dispatch.

10. Reset to Safe Mode - Confirm real-time mode switching capability.

11. Navigate to Modules Page - Walk through all 8 project modules.

12. Open Swagger Docs (http://127.0.0.1:8080/docs) - Demonstrate live API endpoints.

APPENDIX E: HARDWARE COMPONENTS LIST

The following table lists the hardware components planned for future physical prototype implementation. The current software prototype uses simulated sensor data.

S.No.	Component	Specification	Qty	Purpose
1	ESP32 DevKit V1	Dual-core 240MHz, Wi-Fi+BLE	1	Central microcontroller
2	MPU6050 Module	6-axis MEMS accel+gyro	1	Impact/orientation detection
3	MQ-3 Sensor	Semiconductor alcohol sensor	1	Breath alcohol detection
4	SW-420 Sensor	Normally closed vibration	1	Mechanical impact detection
5	NEO-6M GPS	50-channel GPS, UART	1	Real-time location tracking
6	SIM800L Module	Quad-band GSM/GPRS	1	SMS emergency dispatch
7	Active Buzzer	5V piezoelectric	1	Audible warning alerts
8	5V Relay Module	Optocoupler isolated	1	Ignition lock control
9	IR Sensor	Infrared proximity	1	Eye blink/drowsiness
10	Li-Po Battery	3.7V, 2000mAh	1	Power supply
11	Voltage Regulator	AMS1117 3.3V	1	Power regulation
12	Helmet	ISI-certified full-face	1	Physical housing
13	Jumper Wires	M-M, M-F assorted	Set	Sensor connections
14	Breadboard	830-point	1	Prototyping connections
15	USB Cable	Micro-USB	1	Programming and power